

PROMPT — DÉVELOPPEMENT D'UNE PLATEFORME NUMÉRIQUE DE VÉRIFICATION ACADÉMIQUE

1. RÔLE

Tu es un **architecte logiciel senior, développeur PHP full-stack et expert en cybersécurité**.

Ta mission est de concevoir et développer une plateforme web professionnelle permettant aux universités de gérer leurs données académiques officielles et aux entreprises de vérifier rapidement le parcours académique d'un candidat lors d'un recrutement.

Le projet doit être développé principalement en **PHP**, avec une architecture moderne, sécurisée, maintenable et évolutive.

Ne te contente pas de produire une maquette visuelle : je veux une **application fonctionnelle avec base de données, authentification, rôles, API, interface administrateur et interface entreprise**.

2. OBJECTIF DU PROJET

Créer une plateforme centralisée de **vérification et d'authentification des parcours académiques**.

Le système doit permettre à une université de transmettre et gérer ses données académiques officielles, notamment :

- les étudiants ;
- les facultés ;
- les promotions ;
- les années académiques ;
- les palmarès ;
- les diplômes ;
- les numéros/IDs étudiants ;
- les résultats académiques nécessaires à la vérification.

Les entreprises pourront ensuite rechercher un candidat et vérifier son parcours académique sans devoir envoyer physiquement un agent dans l'université.

Le système doit également permettre de générer un **document de vérification académique** contenant :

- l'identité du candidat ;
- son ID étudiant ;
- son université ;
- sa faculté ;
- son année académique ;
- sa promotion ;
- les informations académiques autorisées ;
- un identifiant unique de vérification ;

- un QR Code ;
 - la signature numérique ou électronique de l'autorité universitaire habilitée.
-

3. UTILISATEURS ET RÔLES

Prévoir un système RBAC (Role-Based Access Control).

ROLE 1 — SUPER ADMINISTRATEUR

Il gère toute la plateforme.

Fonctionnalités :

- gérer les universités ;
 - créer/suspendre des comptes universitaires ;
 - gérer les entreprises ;
 - gérer les administrateurs ;
 - consulter les statistiques globales ;
 - superviser les vérifications ;
 - gérer les paramètres de sécurité ;
 - consulter les journaux d'activité ;
 - gérer les permissions.
-

ROLE 2 — ADMINISTRATEUR UNIVERSITAIRE

Chaque université possède son propre espace.

Il peut :

- gérer son établissement ;
- gérer les facultés ;
- créer les années académiques ;
- importer les palmarès ;
- ajouter/modifier les étudiants ;
- enregistrer les diplômes ;
- valider les informations académiques ;
- gérer les promotions ;
- générer les documents officiels ;
- gérer l'autorité signataire ;
- consulter l'historique des vérifications concernant son université.

Important :

Une université ne doit jamais pouvoir accéder aux données d'une autre université.

ROLE 3 — ENTREPRISE

L'entreprise dispose d'un espace professionnel sécurisé.

Elle peut :

- se connecter ;
- rechercher un candidat ;
- sélectionner une université ;
- filtrer les résultats ;
- consulter les informations académiques autorisées ;

- effectuer une vérification ;
- générer un reçu/certificat de vérification ;
- télécharger/imprimer le document ;
- consulter son historique de vérifications.

ROLE 4 — ÉTUDIANT / DIPLÔMÉ

Prévoir éventuellement un espace étudiant permettant de :

- consulter son profil académique ;
- consulter son ID étudiant ;
- consulter ses diplômes validés ;
- consulter ses documents de vérification ;
- générer ou partager un lien de vérification ;
- vérifier l'authenticité de ses documents.

4. AUTHENTIFICATION

Créer une page de connexion professionnelle.

Prévoir :

- email/identifiant ;
- mot de passe ;
- récupération du mot de passe ;
- gestion des sessions ;
- déconnexion ;
- protection CSRF ;
- limitation des tentatives de connexion ;
- hash sécurisé des mots de passe ;
- gestion des rôles ;
- journalisation des connexions.

Prévoir une architecture permettant d'ajouter ultérieurement une authentification à deux facteurs (2FA).

5. INTERFACE UNIVERSITÉ

Créer un dashboard moderne.

Tableau de bord

Afficher :

- nombre total d'étudiants ;
- nombre de diplômés ;
- nombre de facultés ;
- nombre de palmarès enregistrés ;
- nombre de diplômes vérifiés ;
- nombre de vérifications effectuées ;
- statistiques par année académique ;
- dernières activités.

Gestion des étudiants

Créer une interface CRUD :

- Ajouter étudiant
- Modifier étudiant
- Consulter étudiant
- Désactiver étudiant
- Rechercher étudiant

Informations possibles :

- ID unique étudiant ;
- matricule ;
- nom ;
- postnom ;
- prénom ;
- date de naissance si nécessaire ;
- faculté ;
- département ;
- promotion ;
- année académique ;
- niveau ;
- statut académique ;
- diplôme obtenu ;
- date d'obtention.

6. IMPORTATION DES PALMARÈS

Fonction essentielle.

L'université doit pouvoir importer les palmarès de chaque année académique et de chaque faculté.

Formats à prévoir :

- CSV ;
- Excel/XLSX.

Exemple :

Université

→ Faculté

→ Année académique

→ Promotion

→ Palmarès

Le système doit proposer une étape de **prévisualisation avant importation définitive**.

Exemple :

1. Upload du fichier
2. Lecture du fichier
3. Affichage des données
4. Détection des erreurs
5. Détection des doublons
6. Validation
7. Importation dans la base de données

8. Rapport d'importation

Le rapport doit indiquer :

- nombre de lignes importées ;
- lignes rejetées ;
- doublons ;
- erreurs ;
- raisons des erreurs.

Ne jamais supprimer automatiquement les données existantes lors d'un nouvel import.

7. RECHERCHE DES CANDIDATS

L'entreprise doit disposer de deux systèmes de recherche.

RECHERCHE DÉTAILLÉE

Étapes :

Université

→ Année académique

→ Faculté

→ Nom du candidat

Exemple :

Université : UNIKIN

Année académique : 2024-2025

Faculté : Sciences économiques

Nom : MBOMBO

Le système affiche les candidats correspondants.

RECHERCHE RAPIDE

L'entreprise sélectionne simplement :

Université

puis saisit :

Nom / prénom / matricule

Le système effectue automatiquement le filtrage.

Prévoir :

- recherche insensible aux majuscules/minuscules ;
 - recherche par nom ;
 - recherche par prénom ;
 - recherche par matricule ;
 - recherche partielle ;
 - pagination ;
 - filtres ;
 - gestion des homonymes.
-

8. FICHE DE VÉRIFICATION DU CANDIDAT

Lorsqu'un candidat est trouvé, afficher une fiche claire.

Exemple :

NOM : MBOMBO

PRÉNOM : JÉRÉMIE

ID ÉTUDIANT : UNI-2025-000123

UNIVERSITÉ : XXXXX

FACULTÉ : XXXXX

DÉPARTEMENT : XXXXX

PROMOTION : XXXXX

ANNÉE ACADÉMIQUE : 2024-2025

DIPLÔME : XXXXX

STATUT : VÉRIFIÉ

Ne montrer à l'entreprise que les données autorisées par la politique de confidentialité.

9. DOCUMENT DE VÉRIFICATION

Créer une fonctionnalité :

GÉNÉRER LE DOCUMENT DE VÉRIFICATION

Le document doit contenir :

- logo de l'université ;
- nom officiel de l'université ;
- identité du candidat ;
- ID étudiant ;
- faculté ;
- département ;
- promotion ;
- année académique ;
- diplôme ;
- date de génération ;
- identifiant unique du document ;
- QR Code ;
- signature de l'autorité universitaire ;
- mention indiquant que le document est vérifiable en ligne.

Le document doit pouvoir être généré en **PDF**.

10. QR CODE

Chaque document généré doit posséder un QR Code unique.

Lorsque quelqu'un scanne le QR Code, il est redirigé vers une page publique de vérification.

Exemple :

/verify/XXXXXXXXX

La page doit afficher :

DOCUMENT AUTHENTIQUE

Nom du candidat

Université

Faculté

Année académique

ID du document

Date de vérification

Statut :

✓ AUTHENTIQUE

ou

× DOCUMENT NON VALIDE

Le QR Code ne doit pas contenir directement les informations personnelles du candidat.

Il doit contenir uniquement un identifiant sécurisé permettant au serveur de récupérer les informations nécessaires.

11. SIGNATURE DU RECTEUR

Prévoir dans l'espace universitaire une section :

Autorité signataire

Permettre à l'université de définir :

- nom du recteur/autorité ;
- fonction ;
- signature ;
- date de validité ;
- statut de la signature.

Lorsqu'un document est généré, le système doit utiliser la signature enregistrée.

Prévoir une architecture permettant de remplacer la signature lorsqu'un nouveau recteur prend ses fonctions.

IMPORTANT :

La signature doit être protégée et ne doit pas être accessible directement depuis une URL publique.

12. SYSTÈME DE VÉRIFICATION

Chaque vérification doit générer un événement dans les logs.

Enregistrer :

- entreprise ;
- utilisateur ;
- candidat recherché ;
- université ;
- date ;
- heure ;

- type de vérification ;
- document généré ;
- identifiant de vérification.

L'entreprise doit pouvoir consulter son historique.

13. BASE DE DONNÉES

Utiliser **MySQL/MariaDB**.

Prévoir une architecture relationnelle propre.

Tables principales possibles :

users

roles

universities

faculties

departments

academic_years

students

promotions

degrees

palmares

student_academic_records

verification_documents

verification_logs

digital_signatures

companies

company_users

audit_logs

sessions

password_resets

permissions

role_permissions

Ne pas créer inutilement des tables redondantes.

Utiliser :

- clés primaires ;
- clés étrangères ;
- index ;
- contraintes d'unicité ;
- timestamps ;
- soft delete lorsque pertinent.

14. TECHNOLOGIES

Utiliser :

Backend

PHP 8.3+

Privilégier **Laravel**, sauf raison technique sérieuse de ne pas l'utiliser.

Base de données

MySQL 8+

Frontend

Blade + Tailwind CSS

ou

Blade + Bootstrap.

Choisir une seule approche et rester cohérent.

JavaScript

JavaScript moderne.

Possibilité d'utiliser Alpine.js pour les interactions simples.

PDF

Utiliser une bibliothèque PHP fiable pour générer les documents PDF.

QR Code

Utiliser une bibliothèque PHP reconnue pour générer les QR Codes.

15. ARCHITECTURE

Utiliser une architecture propre inspirée MVC.

Séparer clairement :

- Models
- Controllers
- Services
- Repositories si nécessaire
- Requests
- Policies
- Middleware
- Jobs
- Events
- Notifications
- Views
- API

La logique métier ne doit pas être entièrement placée dans les Controllers.

Créer notamment des services tels que :

AcademicVerificationService

PalmaresImportService

VerificationDocumentService

QRCodeService

SignatureService

AuditLogService

16. SÉCURITÉ

La sécurité est une priorité absolue.

Mettre en place :

- protection CSRF ;
- protection XSS ;

- validation serveur ;
- requêtes SQL préparées/Eloquent ;
- hash Argon2id ou bcrypt ;
- contrôle d'accès par rôle ;
- Politiques Laravel ;
- rate limiting ;
- protection contre les attaques par force brute ;
- sessions sécurisées ;
- HTTPS en production ;
- validation des fichiers uploadés ;
- limitation des extensions ;
- limitation de taille des fichiers ;
- protection des signatures ;
- audit logs ;
- contrôle strict des permissions.

Les entreprises ne doivent jamais pouvoir accéder aux données brutes de la base de données.

17. MULTI-UNIVERSITÉS

La plateforme doit être conçue dès le départ comme une application **multi-universités**.

Chaque donnée universitaire doit être liée à une université.

Exemple :

student.university_id

faculty.university_id

academic_year.university_id

etc.

Un administrateur universitaire ne doit accéder qu'aux données de son université.

Le Super Admin peut accéder à toutes les universités.

18. API

Prévoir une API REST sécurisée.

Elle pourra être utilisée plus tard par :

- application mobile ;
- systèmes universitaires ;
- systèmes RH ;
- partenaires institutionnels.

Prévoir notamment :

POST /api/auth/login

GET /api/universities

GET /api/universities/{id}/students

GET /api/students/{id}

GET /api/verification/{token}

POST /api/verification

Utiliser une authentification API sécurisée, par exemple Laravel Sanctum.

19. DESIGN UI/UX

Créer une interface professionnelle et institutionnelle.

Le design doit inspirer :

- confiance ;
- sécurité ;
- sérieux ;
- modernité ;
- simplicité.

Prévoir :

Sidebar

Dashboard

Étudiants

Palmarès

Facultés

Années académiques

Diplômes

Vérifications

Documents

Utilisateurs

Paramètres

Logs

Dashboard

Utiliser des cartes statistiques, tableaux, graphiques simples et activités récentes.

L'interface doit être responsive :

- ordinateur ;
- tablette ;
- mobile.

20. WORKFLOW GLOBAL

UNIVERSITÉ

Université crée son compte

↓

Admin universitaire se connecte

↓

Configure facultés

↓

Configure années académiques

↓

Importe les palmarès

↓

Le système vérifie les données
↓
L'université valide
↓
Les données deviennent vérifiables

ENTREPRISE

Entreprise crée son compte
↓
Connexion
↓
Sélectionne une université
↓
Recherche le candidat
↓
Consulte les informations autorisées
↓
Clique sur "Vérifier"
↓
Le système confirme l'identité académique
↓
L'entreprise peut générer le document
↓
Document avec QR Code + signature

TIERS

Scanne le QR Code
↓
Page de vérification
↓
Le serveur vérifie l'identifiant
↓
Affichage du statut
✓ DOCUMENT AUTHENTIQUE
ou
X DOCUMENT INVALIDE

21. IMPORTANT — PROTECTION DES DONNÉES

Le système doit respecter le principe du **minimum de données nécessaires**. Une entreprise ne doit pas avoir accès à toutes les informations académiques ou personnelles simplement parce qu'elle possède un compte. Prévoir un système de permissions permettant de définir précisément ce qu'une entreprise peut voir.

Prévoir également un système de consentement lorsque cela est juridiquement nécessaire.

22. LOGIQUE DE VÉRIFICATION

Ne jamais considérer qu'une simple correspondance de nom constitue une preuve d'identité.

Si plusieurs étudiants ont le même nom :

Afficher les correspondances disponibles avec des informations minimales permettant de distinguer les candidats.

Exemple :

MBOMBO Jérémie

ID : UNI-XXXXX

Faculté : Communication

Promotion : 2025

L'utilisateur doit ensuite sélectionner le candidat approprié selon les informations auxquelles il est autorisé à accéder.

23. STATUTS

Prévoir des statuts standardisés :

ACTIVE

PENDING

VERIFIED

REJECTED

EXPIRED

REVOKED

SUSPENDED

Un document peut notamment être :

VERIFIED

REVOKED

EXPIRED

24. RÉVOCATION

Une université doit pouvoir révoquer un document précédemment généré.

Si quelqu'un scanne ensuite son QR Code, le système doit afficher :

DOCUMENT RÉVOQUÉ

et non "authentique".

Conserver l'historique de la révocation.

25. AUDIT

Chaque action importante doit être enregistrée :

- connexion ;
- déconnexion ;

- ajout d'étudiant ;
- modification ;
- suppression ;
- importation de palmarès ;
- génération de document ;
- vérification ;
- révocation ;
- modification de signature ;
- changement de permission.

Créer un système d'**Audit Log**.

26. MVP

Ne cherche pas à construire toutes les fonctionnalités avancées immédiatement.

Développe d'abord un MVP fonctionnel comprenant :

Phase 1

Authentification

-

Gestion des utilisateurs

-

Gestion des universités

-

Gestion des facultés

-

Gestion des années académiques

-

Gestion des étudiants

Phase 2

Importation des palmarès

-

Recherche de candidats

-

Vérification académique

Phase 3

Génération PDF

-

QR Code

-

Signature universitaire

-

Page publique de vérification

Phase 4

Dashboard entreprise

•

Historique des vérifications

•

Audit logs

•

Permissions avancées

27. LIVRABLES ATTENDUS

À la fin du développement, fournir :

1. Code source complet
2. Base de données
3. Migrations
4. Seeders
5. Modèles
6. Controllers
7. Services
8. Middleware
9. Policies
10. Routes
11. API
12. Interfaces Blade
13. Système d'authentification
14. Génération PDF
15. Génération QR Code
16. Système de signature
17. Système d'audit
18. Documentation d'installation
19. Documentation API
20. Guide utilisateur

28. MÉTHODE DE DÉVELOPPEMENT

Ne génère pas toute l'application en une seule réponse.

Travaille étape par étape.

Avant chaque phase :

1. Explique brièvement ce que tu vas construire.
2. Identifie les fichiers concernés.
3. Implémente la fonctionnalité.
4. Vérifie les dépendances.
5. Vérifie les erreurs potentielles.
6. Effectue les tests nécessaires.
7. Passe à la phase suivante uniquement lorsque la précédente est cohérente.

Ne supprime jamais une fonctionnalité existante sans raison.

Lorsque tu modifies un fichier existant, préserve les fonctionnalités déjà opérationnelles.

29. PREMIÈRE ÉTAPE

Commence par analyser l'ensemble de ce cahier des charges.

Ensuite :

1. Propose l'architecture technique.
2. Propose le schéma complet de la base de données.
3. Présente les relations entre les tables.
4. Présente l'arborescence du projet Laravel.
5. Identifie les packages nécessaires.
6. Identifie les risques de sécurité.
7. Propose le plan de développement par étapes.

Ne commence pas immédiatement à générer tous les fichiers.

Commence par cette analyse technique et attends ma validation avant de passer à l'implémentation.